

CourtLink - A Web-Based Legal Assistance Platform

Team Members:

Diya George (FIT23CS076)

Diya K D (FIT23CS077)

Hannah Tresa Renjith (FIT23CS092)

Krishna S Laiju (FIT23CS124)

Review 3



Contents

1	Introduction
2	Problem Statement
3	Objectives
4	Purpose and Need
5	Scope of the project and Social Relevance
6	UN SDG Alignment
7	Software Requirement Specification (SRS) ● Functional Requirements ● Non-Functional Requirements
8	System Architecture
9	Application Architecture Design
10	GUI Design
11	API Design
12	Database Diagram
13	Technology Stack
14	Use Cases / DFD
15	Algorithms
16	Development Practices
17	Testing
18	Deployment
19	Individual Contributions
20	Screenshots
21	Project Timeline
22	Conclusion

Introduction

- The Indian judicial system is gradually adopting digital solutions to improve accessibility, transparency, and efficiency, but existing platforms often lack user-friendly interfaces.
- CourtLink is a web-based platform designed to provide a centralized and intuitive system for accessing legal support services.
- It enables users to file case requests, track case progress, manage documents, and connect with advocates efficiently.
- The system enhances access to justice, improves communication, and supports digital governance aligned with SDG 16.

Problem Statement

- Existing judicial platforms are complex and difficult for general users to navigate.
- Lack of integration between case tracking, document management, and advocate services reduces efficiency.
- Limited accessibility for non-technical users creates a barrier to legal support.
- Difficulty in effectively connecting citizens with legal professionals.
- CourtLink addresses these issues by providing a unified, user-friendly platform integrating case management, document handling, and advocate connectivity.

Technical Objectives

- Develop a user-friendly platform for accessing judicial services
- Provide secure role-based access (Client, Advocate, Admin)
- Enable case request submission and document management
- Support advocate assignment and case tracking
- Integrate AI features (Legal Assistant, Document Summarizer)
- Implement secure online payments (Razorpay)

Societal & Sustainability Objectives

- Improve accessibility to legal services
- Reduce paperwork through digital solutions
- Promote digital governance and access to justice (SDG 16)

Purpose and Need

Purpose

- Develop a centralized platform to simplify interaction between clients and advocates
- Enable efficient and accessible legal support services

Need

- Simplifies how citizens request legal assistance and connect with advocates
- Reduces paperwork through digital document management
- Improves communication between clients and legal professionals
- Provides intelligent tools to assist in understanding legal information

Scope of the Project and Social Relevance

Scope of the Project

- Role-based authentication (Client, Advocate, Admin)
- Case request submission and tracking
- Legal document upload and management
- Advocate assignment and connectivity
- AI features: Legal Assistant & Document Summarizer
- Secure payment integration (Razorpay)

Social Relevance

- Improves access to legal services and reduces physical visits
- Enables secure digital document handling and reduces paperwork
- Enhances understanding through AI-assisted legal tools
- Promotes efficient communication and digital justice (SDG 16)

Alignment with UN Sustainable Development Goals

- Relevant primary and secondary SDGs are identified
- The project contributes to improving accessibility, infrastructure, and inclusivity

SDG No.	SDG Title	Contribution
16	Peace, Justice & Strong Institutions	Improves access to justice
9	Industry, Innovation & Infrastructure	Supports digital judicial infrastructure
10	Reduced Inequalities	Ensures equal access to legal services

Software Requirement Specification (SRS)

- The SRS defines the functional and non-functional requirements of the CourtLink system
- CourtLink is a web-based legal assistance platform that supports case management, document handling, advocate coordination, and secure payments
- The system follows a client–server architecture ensuring scalability and efficient communication
- It includes secure authentication, role-based access control, and integration with external services such as payment gateways and AI APIs
- The SRS acts as a blueprint for system design, development, testing, and validation

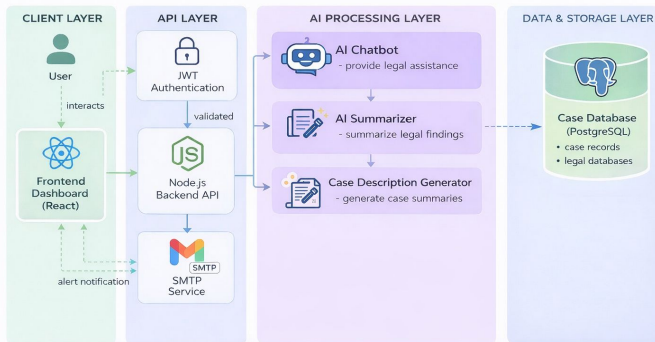
SRS - Functional Requirements

- **User & Access Control:** Registration, login, OTP, role-based access
- **Case Management:** Case request submission and tracking
- **Admin Module:** Case review and advocate assignment
- **Document Management:** Upload, view, and download documents
- **Payments:** Secure transactions using Razorpay
- **Communication:** Notifications and interaction between users
- **AI Features:** Legal assistant and document summarization
- **Consultation:** Request and manage advocate consultations

SRS - Non-Functional Requirements

- **Security:** Authentication and role-based authorization
- **Usability:** Simple and user-friendly interface
- **Performance:** Efficient handling of requests and responses
- **Reliability:** Stable system with consistent database connectivity
- **Compatibility:** Accessible across browsers and devices
- **Scalability:** Supports increasing users and future expansion
- **Integration:** Secure integration with APIs and payment services

System Architecture



CourtLink System Architecture

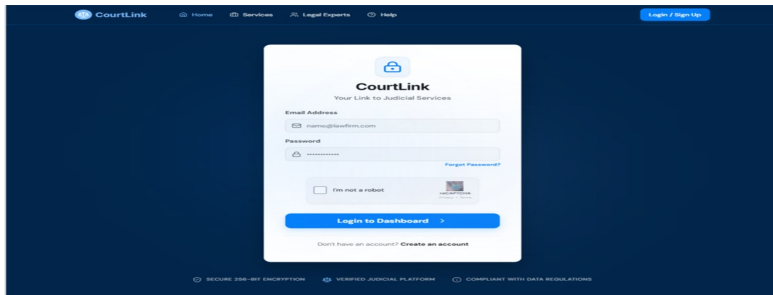
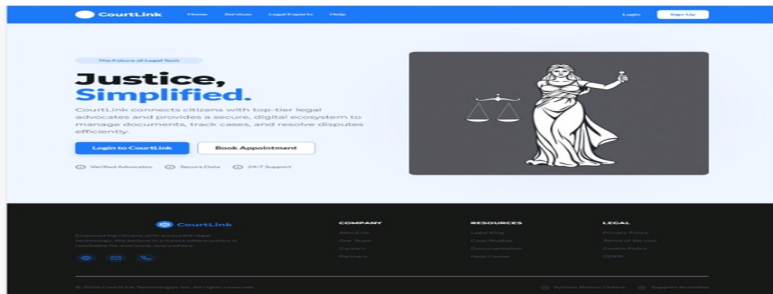
Application Architecture Design

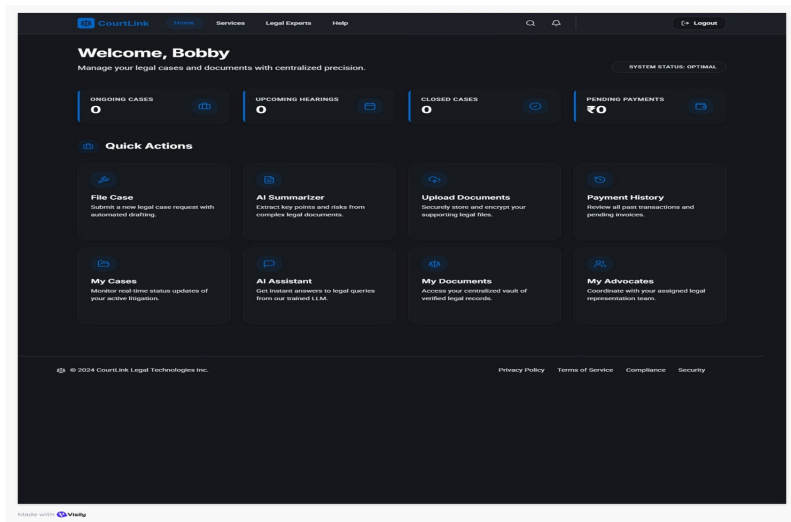
The CourtLink system follows a modular layered architecture with three main components:

- **Frontend:** Built using React.js, it provides user interfaces for clients, advocates, and administrators, supporting actions like login, case filing, booking, and updates through APIs.
- **Backend:** Developed with Node.js and Express.js, it handles authentication, case management, document processing, and payments with secure access using JWT and role-based control.
- **Database:** A relational database stores user, case, document, and transaction data while ensuring consistency and efficient retrieval.

This design ensures smooth interaction, security, and scalability.

GUI Design





GUI Design

File a New Case

Initiate your legal proceedings by providing structured information. Use our AI assistant to help refine your legal narrative.

[Dashboard](#)[My Cases](#)

Case Details

Primary information for legal record

Case Title

e.g. Johnson vs. TechCorp Breach of Contract

Case Category

Select Category

Case Description

Provide a detailed official description of the legal matter...

Minimum 200 words recommended for formal filing.

0 / 5000

Encrypted Submission

Save as Draft

Submit Final Case

ADVANCED AI ACTIVE

AI Assistant

Translate your issue into professional legal terminology instantly.

EXPLAIN IN PLAIN LANGUAGE

e.g. My employer fired me because I told them about safety issues in the warehouse...

Generate Case Narrative

HOW IT WORKS

Our AI analyzes hundreds of successful case filings to suggest high-impact legal phrasing, identifying key facts and legal theories based on your story.

AI CAPABILITIES

Contract Law

Liability Search

Text Analysis

Precedent Match

Upload Documents

Submit your legal filings and case evidence securely.

[Dashboard](#)[My Documents](#)

Upload New Document

Please fill in the details below to associate your document with an active case.

Select Case

-- Select Case --

Choose File

Click to upload or drag and drop

PDF, DOCX, or JPG (max. 10MB)

Document Category

Evidence

Affidavit

Trial

Contract

Motion

Upload Document

By uploading, you confirm this document is accurate and legally permissible.

Team Members: Diya George (FIT23CS076)

CourtLink

Review 3

16 / 46

The system uses RESTful APIs for communication between frontend and backend.

- **Authentication APIs**

- POST /api/auth/register – User signup
- POST /api/auth/login – User login (JWT)

- **Case Management**

- POST /api/cases – File case
- PUT /api/admin/assign-case – Assign case

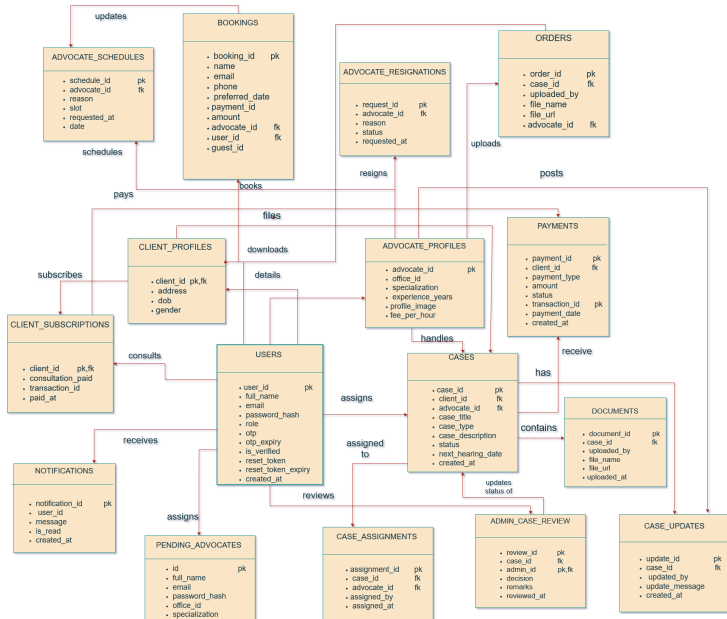
- **Service APIs**

- POST /api/bookings – Book advocate
- POST /api/documents/upload – Upload documents
- POST /api/payments – Payment processing

Key API Details

- **Signup:** Registers new users (client/advocate)
- **Login:** Authenticates users and returns JWT token
- **Case Filing:** Allows users to submit case details
- **Document Upload:** Handles case-related file uploads
- **Booking:** Enables clients to book advocates
- **Payment:** Processes transactions securely
- **Admin Assignment:** Assigns cases to advocates

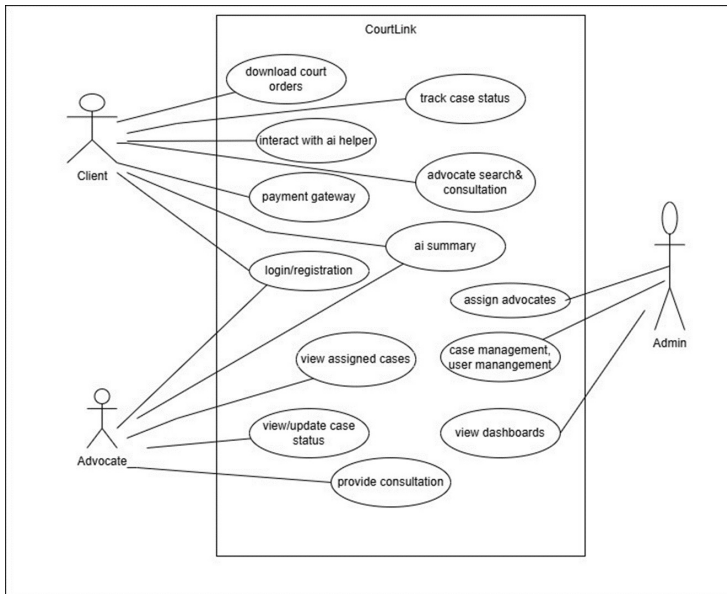
Database Diagram



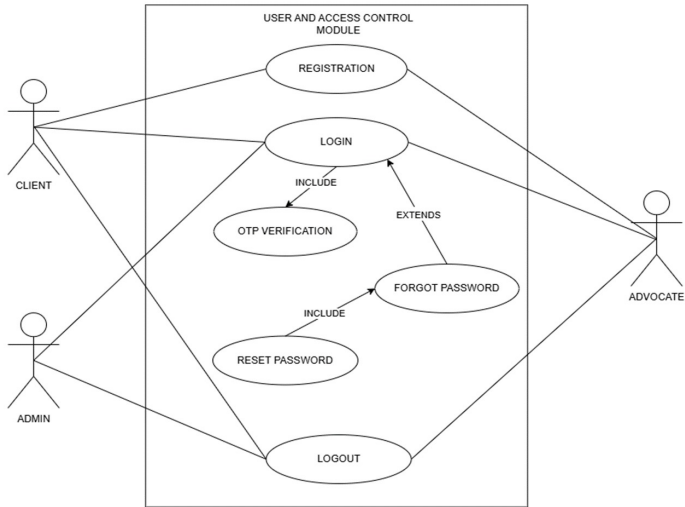
Technology Stack

- **Frontend:** React.js, HTML5, CSS3
 - Provides responsive and interactive user interface
- **Backend:** Node.js with Express.js
 - Handles business logic and API services
- **Database:** PostgreSQL
 - Ensures structured and reliable data storage
- **Tools & Platforms:** Git, GitLab
 - Used for version control and collaboration

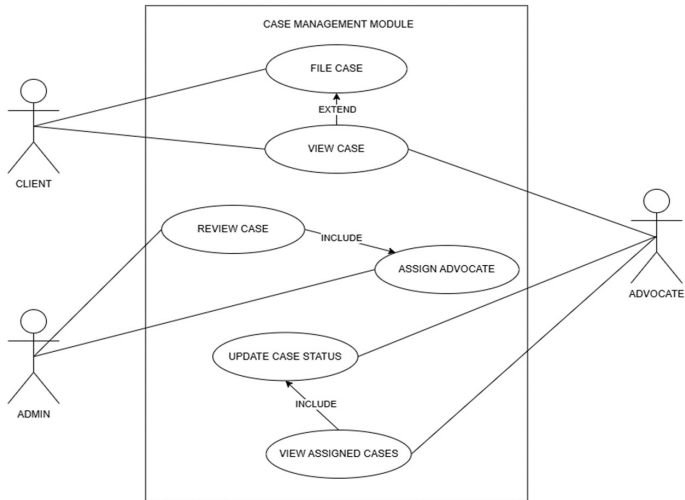
Use Case Diagram 1



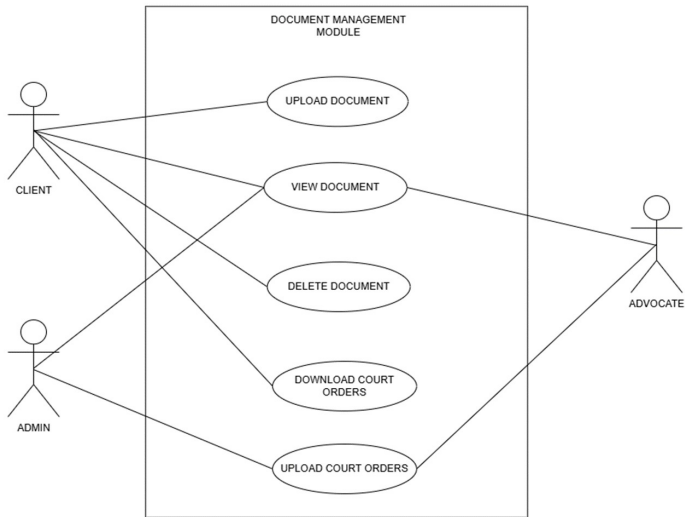
Use Case Diagram 2



Use Case Diagram 3



Use Case Diagram 4



User Authentication

- 1 User enters email and password
- 2 Validate input format
- 3 Check user in database
- 4 If not found → return error
- 5 Compare hashed password
- 6 If mismatch → return error
- 7 Verify OTP (if first login)
- 8 Generate JWT token
- 9 Send token to frontend

AI Query Processing

- 1 User submits query
- 2 Validate non-empty input
- 3 Check legal-domain relevance
- 4 If invalid → return restriction message
- 5 Send query to AI API
- 6 Receive response
- 7 Filter restricted content
- 8 Add disclaimer
- 9 Return response

Document Upload

- 1 User selects case and file
- 2 Validate file and case
- 3 Send POST request to server
- 4 Verify JWT token
- 5 If invalid → return 401
- 6 Process file upload
- 7 Check case ownership
- 8 Store file in server
- 9 Save metadata in database

Subscription Fee Processing

- 1 Client clicks pay
- 2 Check subscription status
- 3 Create Razorpay order
- 4 Send order details
- 5 Open payment gateway
- 6 Complete payment
- 7 Verify payment signature
- 8 If valid → update database
- 9 Enable premium features

User Management

- 1 Admin opens user management
- 2 Retrieve users list
- 3 Select user account
- 4 Activate/deactivate user
- 5 Update database
- 6 Log admin action
- 7 Show confirmation

Coding Standards, Environment Setup Source Code Control

Coding Standards

- Consistent naming conventions
- Modular and reusable code
- Proper indentation and comments
- Effective error handling

Environment Setup

- Node.js and dependencies installed
- PostgreSQL configured
- Environment variables via `.env`

Source Code Control

- Git for version control
- GitLab for repository management
- Branching strategy (main, feature)
- Regular commits and merge requests

Unit Testing

Unit testing was performed using Jest to verify individual components and validation logic.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\admin\courtlink\backend> npx jest tests/unit

  console.log
    Request body: { email: 'test@gmail.com', password: '123', captchaToken: 'token' }
      at Object.log [as login] (src/controllers/auth.controller.js:37:11)

  console.log
    User fetched: { password: 'hashed', is_verified: true, account_status: 'ACTIVE' }
      at Object.log [as login] (src/controllers/auth.controller.js:93:13)

PASS tests/unit/validation.test.js
  ● Console

  console.log
    [dotenv@17.3.1] injecting env (15) from .env -- tip: ✨ run anywhere with `dotenvx run
      at _log (node_modules/dotenv/lib/main.js:139:11)

Test Suites: 2 passed, 2 total
Tests:       17 passed, 17 total
Snapshots:   0 total
Time:        0.867 s, estimated 1 s
Ran all test suites matching tests/unit.
PS C:\Users\admin\courtlink\backend>
```

Integration Testing

Integration testing ensured proper interaction between backend modules and APIs.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\admin\courtlink\backend> npx jest tests/integration

console.log
  LOGIN API HIT

    at log (src/controllers/auth.controller.js:36:11)

console.log
  Request body: { email: 'wrong@gmail.com', password: 'wrongpass' }

    at log (src/controllers/auth.controller.js:37:11)

console.log
  audit_logs table ready

    at log (src/app.js:60:23)

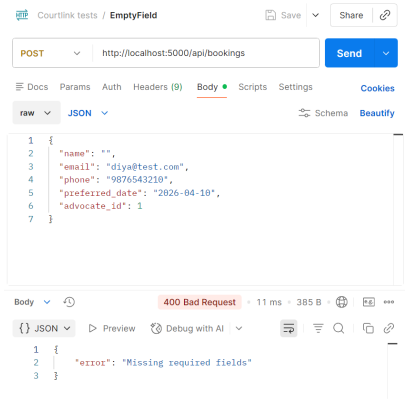
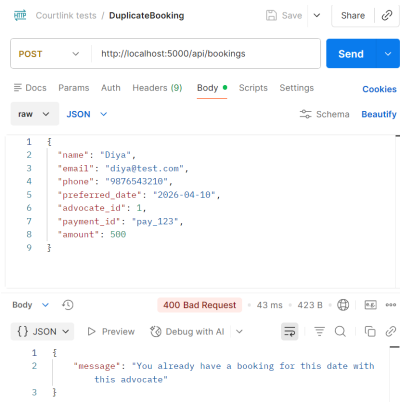
console.log
  User fetched: undefined

    at log (src/controllers/auth.controller.js:93:13)

Test Suites: 4 passed, 4 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        3.032 s
Ran all test suites matching tests/integration.
PS C:\Users\admin\courtlink\backend> |
```

Functional Testing

Functional testing was performed using Postman to validate API responses and workflows.



Load Testing

Load testing using k6 evaluated system performance under concurrent users.

```
JS auth.controll... M
JS payment.controll...
> middleware
  routes
    JS admin.routes.js
    JS advocate.routes.js
    JS ai.routes.js
    JS auth.routes.js
    JS booking.routes.js
    JS clientRoutes.js
    JS order.routes.js
    JS payment.routes.js
  utils
    JS auditLogs.js
    JS mailer.js
    JS razorpay.js
    JS validation.js
  app.js
  db.js
  server.js
  tests
    JS integration
    JS advocate.test.js
    JS auth.test.js
UTLINE

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

TOTAL RESULTS

checks_total.....: 4373 71.280537/s
checks_succeeded....: 100.00% 4373 out of 4373
checks_failed.....: 0.00% 0 out of 4373

✓ login success

HTTP
http_req_duration.....: avg=386.39ms min=198.99ms med=371.75ms max=2.83s p(90)=394.41ms p(95)=422.12ms
{ expected_response:true }....: avg=386.39ms min=198.99ms med=371.75ms max=2.83s p(90)=394.41ms p(95)=422.12ms
http_req_failed.....: 0.00% 0 out of 4373
http_reqs.....: 4373 71.280537/s

EXECUTION
iteration_duration.....: avg=1.38s min=1.2s med=1.37s max=3.83s p(90)=1.39s p(95)=1.42s
iterations.....: 4373 71.280537/s
vus.....: 28 min=28 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 2.6 MB 43 kB/s
data_sent.....: 840 kB 14 kB/s
```

Deployment Details

Frontend

- Deployed using Vercel
- Provides fast and scalable hosting
- Supports continuous deployment from Git

Backend

- Hosted on Node.js server
- Handles API requests and business logic

Database

- PostgreSQL deployed on cloud
- Ensures reliable and scalable data storage

Individual Contributions

Diya George

- Landing page and User authentication module
- Document management module
- Client dashboard, functionalities and AI module

Diya K D

- Authentication features: reCAPTCHA, OTP, forgot password of user authentication module
- Payment integration , deployment
- Admin dashboard structure

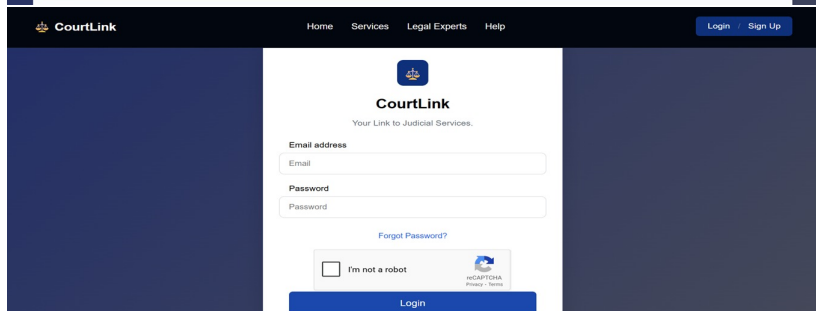
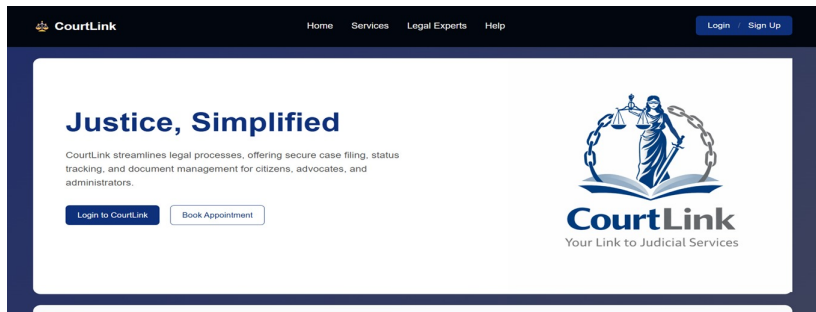
Hannah Tresa Renjith

- Advocate dashboard structure
- Advocate features
- Document management module

Krishna S Laiju

- Admin dashboard features
- Case management module
- Advocate management module

Screenshots of Working Model



Screenshots of Working Model

CourtLink Home Services Legal Experts Help Login / Sign Up

Create Your Account
Join CourtLink to connect with legal professionals or clients.

I am *
Client

Full Name *
Full Name

Phone Number *
+91

Password *
Password

Email Address *
Email

Date of Birth *
dd - mm - yyyy

Confirm Password *
Confirm password

Register

Already have an account? [Login](#)

CourtLink Home Services Legal Experts Help Login / Sign Up

Advocate Registration
Join CourtLink as an Advocate

I am *
Advocate

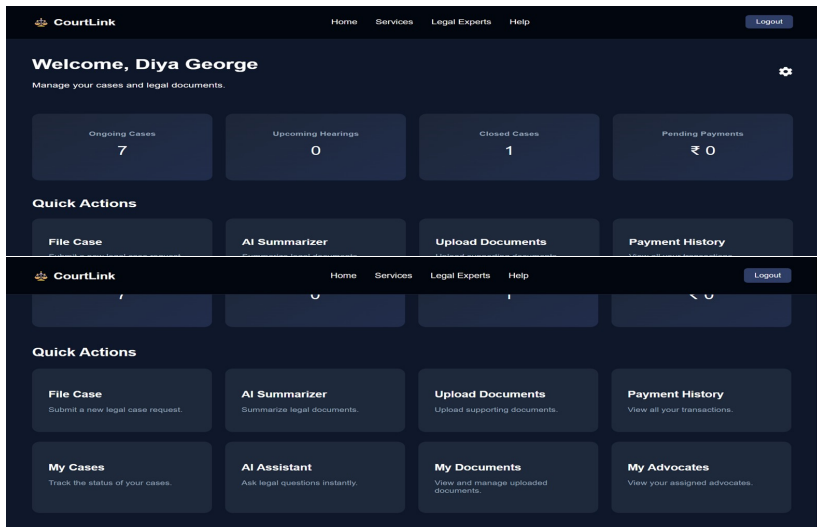
Full Name *
Full Name

Email *
Email

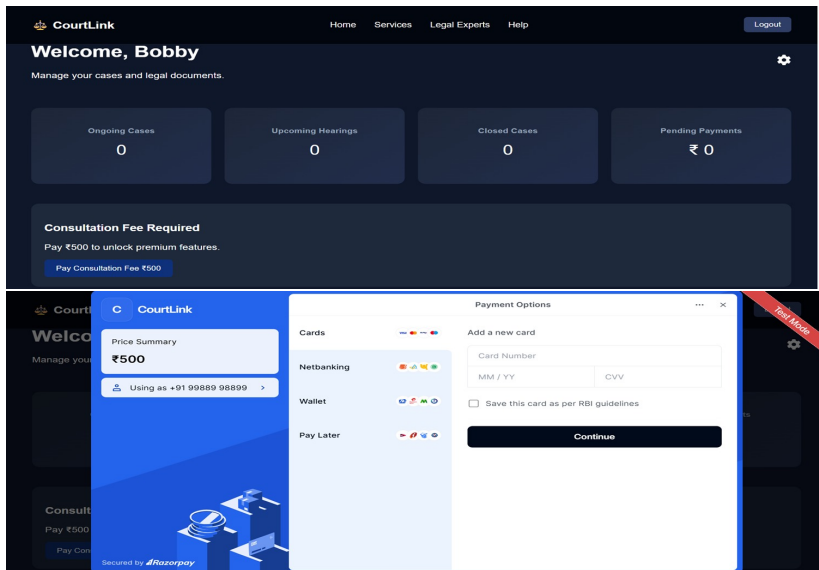
Password *
Password

Confirm Password *
Confirm Password

Screenshots of Working Model



Screenshots of Working Model



Screenshots of Working Model



HomeServicesLegal ExpertsHelp

Logout

File a New Case

+ My Cases

← Dashboard

Case Title

Case Type

Civil / Criminal / Family...

Identify case type

Case Description

Explain your issue briefly...

AI Assistant

Type your issue briefly and AI will draft a professional case description.

Explain your issue...

Generate

Tip: Enter case title + type for best output.



HomeServicesLegal ExpertsHelp

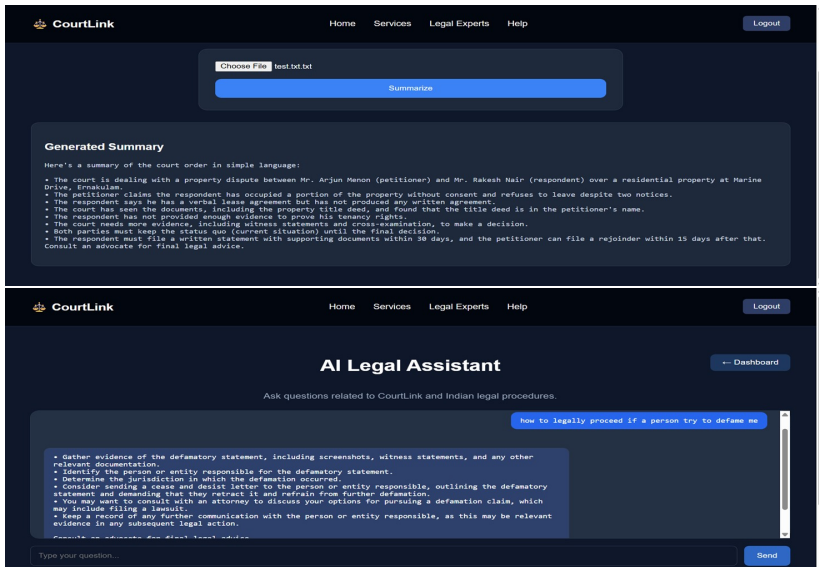
Logout

	Property Boundary Issue	Civil	ONGOING	2/20/2026	2/17/2026
5	Employment Salary Issue	Labour	ONGOING	2/20/2026	2/17/2026
2	Cheque Bounce Complaint	Criminal	APPROVED	2/25/2026	2/17/2026
3	Divorce Filing	Family	PENDING	N/A	2/17/2026

Case Description

Case Description: Background: This case pertains to a dispute over property boundaries between the plaintiff and the defendant, where the defendant is allegedly claiming a portion of the plaintiff's land as their own. Incident Summary: The defendant has been threatening the plaintiff's family and obstructing their access to a shared pathway. Furthermore, the defendant has been attempting to extend their boundary wall into the plaintiff's property, despite the plaintiff's objections. Opposite Party: The opposite party in this case is the defendant, the plaintiff's neighbour, who is claiming ownership of a portion of the plaintiff's land. Relief Requested: The plaintiff seeks relief to protect their property rights, prevent the defendant's unauthorized extension of the boundary wall, and ensure safe and unhindered access to the shared pathway.

Screenshots of Working Model



Screenshots of Working Model

CourtLink Home Legal Experts Help Logout

Admin Dashboard

Total Cases	Active Cases	Closed Cases	Total Advocates
27	3	1	5

Clients	Advocates	Registrations	Case Review
View registered clients	5 registered advocates	Approve or reject advocate registrations	20 cases pending

Case Assignment	Closed Cases	Analytics	Closures
Assign advocates to cases	View completed cases	Visual reports & trends	Approve account closure requests

Recent Activity

Action	Case	By Admin	Time
Case Filed	Overtime work	Diya George	3/24/2026, 12:00:54 PM
Case Filed	Online harassment	Diya George	3/24/2026, 11:58:06 AM

CourtLink Home Legal Experts Help Logout

Welcome Adv. Hannah!
A centralized workspace to manage cases, hearings, and legal responsibilities.

Total Cases	Ongoing Cases	Cases This Week	Total Clients
4	1	0	2

Quick Actions

Add New Case	View My Cases	My Schedules	Upload Order
Create a new case for a client	View all cases assigned to you	Add busy dates and calendar reminders	Upload court orders and judgement documents

Project Timeline and Scheduling

Week	Module / Feature	Tasks / Activities
Week 1	Project Planning & Requirement Analysis	Discuss project idea, define system objectives, identify client/advocate/admin roles, prepare initial requirement list
Week 2	System Design & Database Design	Create ER diagram, design database schema, define relationships between tables, finalize system architecture
Week 3	Authentication Module	Implement user registration, OTP email verification, password hashing using bcrypt, user login functionality
Week 4	JWT Authentication & Client Dashboard	Implement JWT token based login, create client dashboard UI, develop case request form, integrate frontend with backend APIs
Week 5	Advocate Module & Case Management	Create advocate dashboard, implement advocate profile management, implement case listing and case assignment logic
Week 6	Admin Module	Develop admin dashboard, implement client management, advocate management, implement advocate assignment for cases
Week 7	AI Features	Implement AI chatbot for legal assistance, develop AI case summarizer, implement professional case format generator
Week 8	Payment Module	Integrate Razorpay payment gateway, implement payment verification logic, create payment history feature
Week 9	System Integration & Deployment	Deploy database using Neon, deploy backend using Render, configure environment variables and API connections
Week 10	Frontend Deployment & Final Testing	Deploy frontend using Vercel, verify frontend-backend integration, perform payment testing and finalize system

Conclusion

- Developed a complete web-based legal assistance system
- Implemented secure authentication and role-based access
- Integrated key features like case management, document upload, and payments
- Ensured system reliability through testing and validation

References

- Node.js Documentation – <https://nodejs.org/>
- React Documentation – <https://react.dev/>
- Express.js Documentation – <https://expressjs.com/>
- PostgreSQL Documentation – <https://www.postgresql.org/docs/>
- Neon Database – <https://neon.tech/>
- Razorpay Documentation – <https://razorpay.com/docs/>
- Cloudinary Documentation – <https://cloudinary.com/documentation>
- JWT (JSON Web Token) – <https://jwt.io/>
- Google reCAPTCHA – <https://developers.google.com/recaptcha>
- Nodemailer – <https://nodemailer.com/about/>
- Groq AI API – <https://console.groq.com/docs>